

INSTRUÇÕES DE DESVIO NO MIPS

Da estrutura de controle em C para a linguagem de montagem em Assembly

Labels, Program Counter e mudança do fluxo de execução do programa

O QUE SERÁ TRATADO

Resumo

- **Labels** identificam destinos
- **Branches** testam condições
- **Jumps** mudam o fluxo sem testar
- **SLT** constrói comparações

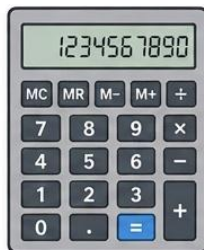
Emprego na tomada de decisões

- Criação de **Switch-Case**
- Implementação de um **for(; ;)** crescente/decrescente, **WHILE**, etc
- Buscar um valor em um **array**

IMPORTÂNCIA DA TOMADA DE DECISÃO

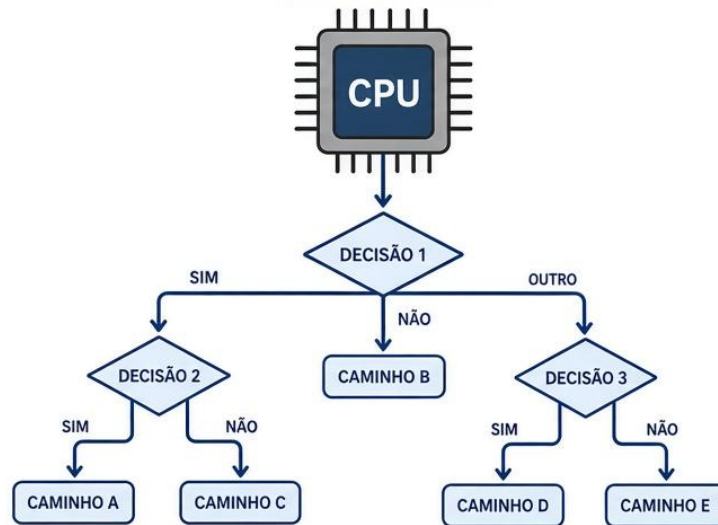
Decisões em Software

CALCULADORA
ELETRÔNICA



ÚNICO CAMINHO
DE EXECUÇÃO (LINEAR)

COMPUTADOR



MÚLTIPLOS CAMINHOS
DE EXECUÇÃO (ESCOLHAS)

“A diferença entre um computador e uma simples calculadora eletrônica é a capacidade do computador de escolher entre os diversos caminhos de execução”.

ALGUMAS DEFINIÇÕES

Os operadores aqui trabalhados tem o mesmo objetivo - produzir um valor representando **Falso** ou **Verdadeiro** que possa ser usado para direcionar o fluxo de um programa

Em C:

Falso - inteiro 0 (zero)

Verdadeiro - inteiro 1 (um)

Exemplo:

```
# define FALSE 0
```

```
# define TRUE 1
```

ALGUMAS DEFINIÇÕES (cont.)

Operadores Relacionais - comparam os valores relativos de operandos

Em C:

> maior que

>= maior ou igual a

< menor que

<= menor ou igual a

Exemplo:

if (a >= b)

<comando se a é maior que b>;

else

<comando se a é menor que b>;

ALGUMAS DEFINIÇÕES (cont.)

Operadores de Igualdade - descobrir se dois valores são iguais

Em C:

== testa se dois valores são iguais

!= testa se dois valores são diferentes

Exemplo:

if (c != d)

<comando c é diferente de d>;

else

<comando c é igual a d>;

ALGUMAS DEFINIÇÕES (cont.)

Operadores Lógicos - forma expressões relacionais complexas

Em C:

usa operadores **&&** (and) e **||** (or)

Exemplo:

if ((c >= d) && (a <= 25))

<comando se a expressão é verdade>;

else

<comando se a expressão é falsa>;

Exemplo em C

Com esses operadores relacionais é possível executar tomada de decisões em software

Exemplo em C:

```
int a, b;  
a = 10;  
b = 20;  
i  f (a < b)  
    printf("a é menor que b");  
else  
    printf("a é maior que b");
```

Exemplo em Assembly MIPS

Com esses operadores relacionais é possível executar tomada de decisões em software

Exemplo em .ASM:

.data

msg_menor: .asciiz "a é menor que b"

msg_maior: .asciiz "a é maior que b"

.text

carrega valores

li \$t0, 10 # \$t0 = a **1a parte**

li \$t1, 20 # \$t1 = b

if (a < b)

slt \$t2, \$t0, \$t1 # \$t2 = 1 se a < b, senão \$t2 = 0

se \$t2 == 0 (a >= b), pula para o else

beq \$t2, \$zero, **else_branch**

bloco if (condição verdadeira)

la \$a0, msg_menor

li \$v0, 4 # syscall 4 imprime string

syscall

j fim # desvia para o fim

else_branch:

bloco else

la \$a0, msg_maior: **2a parte**

li \$v0, 4

syscall

fim:

li \$v0, 10 # syscall 10 = encerrar programa

syscall

Palavras

Entender o significado de algumas palavras ajuda no entendimento do Assembly

BRANCH - derivar, ramo; pode ser entendido como **DESVIAR**

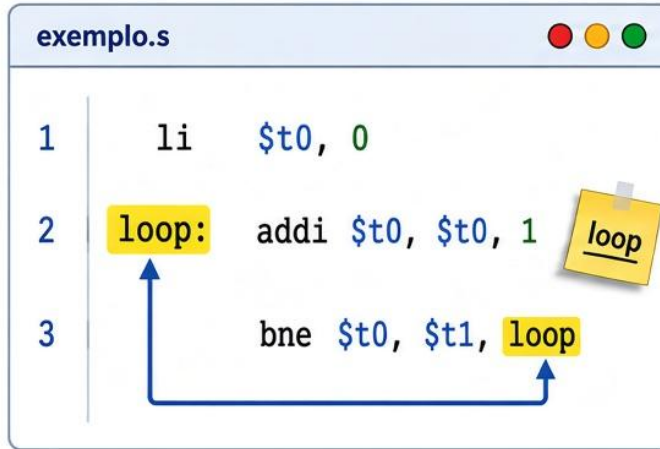
EQUAL - igual; aqui estamos comparando alguma coisa com algo. Vamos usar para comparar valores entre registradores

SET - atribuir valor/constante 1(um). Vamos usar para atribuir valor 1 para um registrador

LESS THAN - menor que. Vamos usar para comparar valor de um registrador

JUMP - saltar, desviar. Vamos usar para fazer saltos para algum endereço

LABELS



Endereços de memória (exemplo)

0x00400000	li \$t0, 0
0x00400004	addi \$t0, \$t0, 1
0x00400008	bne \$t0, \$t1, loop
0x0040000C	...

loop → 0x00400004



Label = um nome dado a um endereço de memória.

Não é uma instrução — o assembler substitui o nome pelo endereço.

LABELS (cont.)

Instruções **FORMATO I** (ex. BNE) não guardam o endereço do label - guardam sim a distância até o label, dividida por 4 - ou seja, o montador “gera” um **OFFSET**

Exemplo de cálculo de OFFSET:

endereço | código

0x00400000: li \$t0, 0

0x00400004: loop: addi \$t0, \$t0, 1

0x00400008: bne \$t0, \$t1, loop

0x0040000c:

Passo 1 - BNE está em PC = 0x00400008

A instrução seguinte (PC+4) é 0x0040000C

Passo 2 - loop aponta para 0x00400004

Passo 3 - cálculo do offset

$(0x00400004 - 0x0040000C) / 4 = -8 / 4 = -2$

Passo 4 - como o Imediato possui 16 bits

-2 -> 1111111111111110 (0xFFFE)

Passo 5 - montar a instrução BNE

Isto vai para
memória

opcode	rs	rt	offset
000101	01000	01001	1111111111111110

LABELS (cont.)

Como o MIPS recalcula o endereço na execução - faz a conta inversa

Endereço alvo = $(PC+4) + (OFFSET \ll 2)$

Com base no exemplo anterior, temos:

$0x0040000C + (-2 \ll 2) = 0x0040000C + (-8) = \mathbf{0x00400004}$



assim o processador descobre qual
endereço ele deve retornar

A FAMÍLIA DE DESVIOS CONDICIONAIS

Instrução	Significado	Equivalente em C
beq rs, rt, L	desvia se iguais/branch if equal	if (rs == rt) goto L;
bne rs, rt, L	desvia se diferentes/branch if not equal	if (rs != rt) goto L;
beqz / bnez	compara com zero/branch if equal zero	if (rs == 0) / if (rs != 0)
bltz / bgez	menor/menor ou igual a zero	if (rs < 0) / if (rs >= 0)
blez / bgtz	menor/igual / maior que zero	if (rs <= 0) / if (rs > 0)

beqz - branch if equal zero - pseudo instr.

bltz - branch if less than zero

bgez - branch if greater than or equal to zero

blez - branch if less than or equal to zero

bgtz - branch if greater than zero

IF: COMPARE E DESVIE

C

```
if (i == j) {  
    f = g + h;  
} else {  
    f = g - h;  
}
```

MIPS

```
bne $s3, $s4, Else  
add $s0, $s1, $s2  
j Exit  
Else: sub $s0, $s1, $s2  
Exit:
```

Testa a condição oposta para saltar o bloco que não será executado

COMPARADORES

O MIPS constrói comparações complexas colocando 0 ou 1 em um registrador

Instrução	Significado	Formato
SLT/SLTU	set less than / unsigned	R
SLTI/SLTIU	set less than immediate / unsigned	I

COMPARADORES (cont.)

Exemplo:

Quando não há um branch direto, transforme a comparação em 0 ou 1 com **slt**

C

```
if (a < b)
{
    bloco;
}
```

A condição é booleana.

MIPS

```
slt $t0, $s0, $s1
bne $t0, $zero, L1
```

Variações: **slti**, **sltu** e **sltiu**.

FOR, DO-WHILE, BREAK E CONTINUE

for

```
for (i=0; i<n; i++) corpo;
```

Inicialização → teste → corpo →
incremento → salto

do-while

```
do { corpo; } while (cond);
```

O corpo executa antes do primeiro
teste

break

```
if (cond) break;
```

Branch para **Exit_Loop**

continue

```
if (cond) continue;
```

Branch para o teste ou incremento.

FOR (; ;)

Em C

for

```
for (i=0; i<n; i++) corpo;
```

Inicialização → teste → corpo →
incremento → salto

Em Assembly

```
li $t0, 0    # i = 0
              # $t1 já possui N
LOOP:
    bge $t0, $t1, FIM    # if (i >= N) sai do laço
    # corpo do laço
    .....
    .....

    addi $t0, $t0, 1      # i++
    j    LOOP
FIM:
    .....
```

DO-WHILE

Em C

do-while

```
do { corpo; } while (cond);
```

O corpo executa antes do primeiro teste

Em Assembly

LOOP:

corpo do laço

avalia a condição e coloca em \$t2

repete se a condição for verdadeira

bne \$t2, \$zero, LOOP

FIM:

WHILE

Em C

do-while

```
while (cond) { corpo; } ;
```

O teste executa antes do corpo da condição

Em Assembly

WHILE:

```
# if (cond == 0)
beq $t0, $zero, FIM sai
# corpo da condição
```

```
j WHILE
```

FIM:

BREAK

Em C

break

```
if (cond) break;
```

Branch para **Exit_Loop**

Em Assembly

LOOP:

```
bge $t0, $t1, FIM           # Teste normal do laço  
beq $t3, $zero, SEM_BREAK  # if (!cond) pula o break  
j    FIM                    # break - sai imediatamente
```

SEM_BREAK:

```
# resto do corpo do laço
```

.....

```
addi $t0, $t0, 1
```

```
j    LOOP
```

FIM:

Em C

break

```
if (cond) continue;
```

Branch para o teste ou incremento

Em Assembly

LOOP:

```
bge $t0, $t1, FIM
```

```
# avalia a condição do continue
```

```
bne $t3, $zero, PROX # if (cond) continue → pula o resto
```

```
# resto do corpo do laço
```

PROX:

```
addi $t0, $t0, 1 # i++ ← alvo do continue
```

```
j LOOP
```

FIM:

SALTOS E CHAMADAS DE FUNÇÃO

Saltos incondicionais mudam o PC sem testar uma condição

Instrução	Uso no MIPS	Correspondente em C
j Label	salto para um rótulo	goto Label;
jal Rotina	salta e salva retorno em \$ra	chamada de função
jr \$ra	retorna ao chamador	return;
jr \$t0	salta para endereço em registrador	ponteiro de função

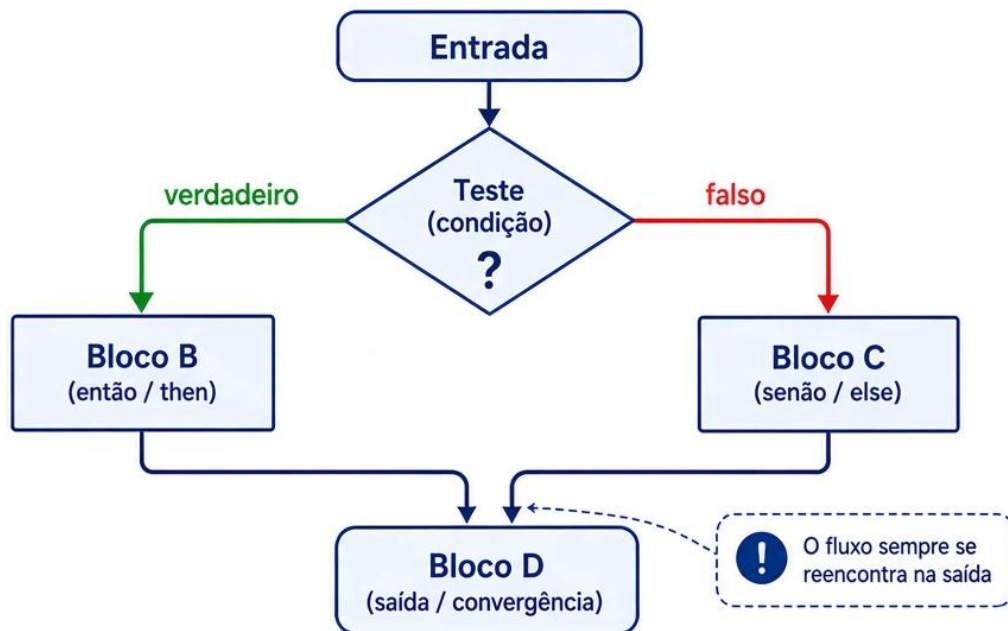
REPRESENTAÇÃO DOS ENDEREÇOS DE DESVIO

Formato	Campos	Destino
Branch — Tipo I	op, rs, rt, offset (16)	PC + 4 + (offset << 2)
Jump — Tipo J	op, target (26)	PC [31:28] (target << 2)

As instruções têm 4 bytes: os dois bits menos significativos são 00.

O branch é relativo ao PC e alcança uma região próxima. O jump usa 26 bits e permanece dentro de um bloco de 256 MB.

Estrutura if else



Um bloco básico de decisão: após o teste, os caminhos então/senão convergem de volta para um único fluxo de execução, sem saídas escondidas no meio.

EXEMPLO: SOMA CONDICIONAL EM UM ARRAY

Em C

```
int sum = 0;
for (int i=0; i<n; i++)
{
    if (arr[i] > 10)
    {
        sum += arr[i];
    }
}
```

Laço externo e decisão interna.

MIPS Assembly

```
li $s0, 0
li $t0, 0
Loop:
    bge $t0, $s1, END
    sll $t1, $t0, 2
    add $t1, $t1, $s2
    lw $t2, 0($t1)
    li $t3, 10
    ble $t2, $t3, skip
    add $s0, $s0, $t2
skip:
    addi $t0, $t0, 1
    j Loop
END:
```

EXERCÍCIOS 1

RESPOSTA

1. O que é um label no assembly MIPS? Por que ele "não é uma instrução".
2. Qual é a diferença entre branch e jump?
3. Por que o MIPS não tem uma instrução direta de "desviar se menor que" (`if (a < b)`)? Como a instrução SLT resolve essa limitação e cite as suas variações (`slti`, `sltu`, `sltiu`).
4. "A diferença entre um computador e uma simples calculadora eletrônica é a capacidade do computador de escolher entre os diversos caminhos de execução" (Patterson & Hennessy). Relacione essa frase com as instruções `branch`, `jump` e `slt`, explicando como cada uma contribui para a "escolha de caminhos

EXERCÍCIOS 2

DADO O CÓDIGO

```
0x00400000: li    $t0, 0
0x00400004: loop: addi $t0, $t0, 1
0x00400008: bne   $t0, $t1, loop
0x0040000C: ...
```

Calcule o offset da instrução bne. Por que o resultado é dividido por 4?
Represente o offset em 16 bits.

EXERCÍCIOS 3

CODIFIQUE

1. Traduza o trecho abaixo para MIPS:

```
if (i == j) {  
    f = g + h;  
} else {  
    f = g - h;  
}
```

2. Traduza o laço for para assembly

```
for (i = 0; i < n; i++) {  
    /* corpo do laço */  
}
```

EXERCÍCIOS 4

CODIFIQUE

1. Escreva um programa em MIPS que some os inteiros de 1 até N e armazene o resultado na memória. Considere $N = 10$.
2. Escreva um programa em MIPS que percorra um vetor de inteiros e encontre o maior valor, armazenando-o na memória.
3. Escreva um programa em MIPS que calcule o fatorial de N e armazene o resultado na memória. Considere $N = 5$.